

Python 34 的线程安全

线程安全

2026 年 6 月 28 日

摘要

本文主要介绍了 Python 34 的线程安全。Python 34 的线程安全是通过 GIL (Global Interpreter Lock) 实现的。GIL 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，GIL 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

本文主要介绍了 Python 34 的线程安全。COR 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，COR 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

1. 线程安全

线程安全是指多线程程序在并发执行时，不会因为线程之间的竞争而导致数据不一致或程序崩溃。在多线程编程中，线程安全是一个非常重要的概念。本文将介绍 Python 34 的线程安全实现。

2026 年 4 月，Python 34 的线程安全实现进行了重大改进。在 Python 34 之前，Python 的线程安全是通过 GIL (Global Interpreter Lock) 实现的。GIL 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，GIL 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

在 Python 34 中，线程安全是通过 COR (Coroutine Lock) 实现的。COR 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，COR 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

在 Python 34 中，线程安全是通过 COR (Coroutine Lock) 实现的。COR 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，COR 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

在 Python 34 中，线程安全是通过 COR (Coroutine Lock) 实现的。COR 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，COR 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

在 Python 34 中，线程安全是通过 COR (Coroutine Lock) 实现的。COR 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，COR 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

在 Python 34 中，线程安全是通过 COR (Coroutine Lock) 实现的。COR 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，COR 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

2. 线程安全

线程安全是指多线程程序在并发执行时，不会因为线程之间的竞争而导致数据不一致或程序崩溃。在多线程编程中，线程安全是一个非常重要的概念。本文将介绍 Python 34 的线程安全实现。

2.1 线程安全

2026 年 3 月，Python 34 的线程安全实现进行了重大改进。在 Python 34 之前，Python 的线程安全是通过 GIL (Global Interpreter Lock) 实现的。GIL 是一个全局锁，它确保了在任何时刻，只有一个线程可以执行 Python 字节码。这保证了线程安全，但也带来了性能上的瓶颈。在多线程应用中，GIL 会导致线程之间的竞争，从而降低程序的执行效率。为了解决这个问题，Python 34 引入了线程局部存储 (TLS) 的概念。TLS 允许每个线程拥有自己的局部变量，从而避免了全局变量的竞争。此外，Python 34 还引入了原子操作 (atomic operations) 的概念，用于在多线程环境中进行安全的变量操作。这些改进使得 Python 34 在多线程应用中具有更好的性能和可扩展性。

[illegible]

2.13 34

34

“我對你說的每一句話，都是事實”——這是我在過去幾十年裡最常被問到的問題。18

我對你說的每一句話，都是事實”——這是我在過去幾十年裡最常被問到的問題。

“”——34World

34

3. □□□□

3.1 □□□□

```
import numpy as np

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-8)

class Character:
    def __init__(self):
        self.pool = 0.5
        self.memories = []

    def tick(self, event_emb, intensity, base_drain=0.05, base_recovery=0.02):
        drain = base_drain
        matched = []

        for m in self.memories:
            sim = cosine_sim(event_emb, m["emb"])
            threshold = 0.6 - m["intensity"] * 0.4
            if sim > threshold:
                drain += m["intensity"] * sim * 0.4
                matched.append(m)

        for i in range(len(matched)):
            for j in range(i + 1, len(matched)):
                drain += cosine_sim(matched[i]["emb"], matched[j]["emb"]) * 0.1

        self.pool = max(0, min(1, self.pool - drain + base_recovery))

        if intensity > 0.5:
            self.memories.append({"emb": event_emb, "intensity": intensity})

        return self.pool
```

34□□

3.2 实验

1111

- `self.pool[0, 1]`

- `self.memories` — emb `intensity` [0, 1]

`tick`

- 1 `0.6 - intensity × 0.4`
`intensity` — `intensity=0.9`
`intensity=0.5` `0.4` `"intensity × sim × 0.4"`
- 2 `sim × 0.1` `"` — `"`
- 3 **COR** `pool = pool - drain + recovery` `base_drain=0.05` + `base_recovery=0.02` `clamp[0, 1]`

`0.5` `"` — `"`

3.3

NumPy	dot product norm	Python
		34

3.4

pool	0.5		
base_drain	0.05		
base_recovery	0.02		
	0.6		
	0.4	intensity →	
	0.5		

3.5

	—
	—
thickness/integrity	—
	—
	—

パラメータ	説明
crack_mode	クラックモード。0: 強度のみを評価する。1: 強度と変位を評価する。
domain_sensitivity	領域感度。0: 領域感度を無視する。1: 領域感度を考慮する。
...	...
...	...

3.6 九九九九九九九九

“我這人，就是個‘老好人’，別人有什麼事，我都幫着辦，別人有什麼話，我都聽着說。別人有什麼困難，我都幫着解決。別人有什麼煩惱，我都幫着排解。別人有什麼快樂，我都幫着分享。別人有什麼痛苦，我都幫着承擔。別人有什麼危險，我都幫着保護。別人有什麼秘密，我都幫着保守。別人有什麼心事，我都幫着隱藏。別人有什麼煩惱，我都幫着排解。別人有什麼困難，我都幫着解決。別人有什麼話，我都聽着說。別人有什麼事，我都幫着辦。我這人，就是個‘老好人’。”

[illegible]

“我从来没有想过要当什么大人物，我只想当一个好人。一个对得起自己良心的人。一个对得起这个社会的人。一个对得起这个国家的人。一个对得起这个民族的人。一个对得起这个时代的中国人。”

```
threshold = 0.6 - m["intensity"] * 0.4
intensity=0.9
sim>0.24
intensity=0.5
sim>0.4
```

0.5

3.7

pattern" 34

- intensity trauma_cluster
- intensity "0" "1"
- pool "0" "1"
- "0" —

4.

34

4.1 `Character` → class `Character`

[illegible]

4.2 $\lambda\mu\kappa \rightarrow \text{pool} + \text{memories}$

```
"pool"pool——memories
——poolmemories
——poolmemories——"
```

4.3 `tick()` 関数

```
"pool"[]"[]"[][][][][][][][]"[]"[][][][][]—[] [] [] [] [] [] [] [] [] [] [] [] "[]" [] —[] [] [] [] [] []
[] 34 [] [] [] [] "[]" [] [] [] [] —tick() [] [] [] [] [] [] [] [] [] [] "[]" [] [] [] [] pool [] memories [] [] [] "[]"
[] [] [] [] [] [] [] []
```

4.4 4 → 16

[illegible]

4.5 池化 → pool - drain + recovery

[illegible]

☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ " ☐ ☐ " ☐ ☐

"0000000000"—0000000000000000"000000"—000000000000³⁴0000000000000000"00"
0"00"0"00"0000000000000000—pool=0.1000000000pool=0.9000000000"0000"

Figure 7-6 shows how you can make the program work as intended.

□ □ □ □ □ □ □ □ □ □

"oooooooooooo"—ooooooooooooooooooooooo³⁴oooooooooooooooooooooooooooooooooooo—"oooo
ooooooooooooooooooooooooooooo"o"ooooooooooooooooooooo"oooo"—"oooooooooooooooooooo"oooooooooooooooo
"ooo"

5. ☐ ☐

5.1

[illegible][illegible][illegible]

5.2 □□□□□□□□

□□□□□□□□□□34□□□□□□□□□□510□□

345101472510

1472"thicknessintegritybaselinecrack_mode
EE"

34"tick"

	nuwa.py	68	34tick + (3) + (11) + Character(20) — import
IO	fileio.py	95	heart.yaml / config.yaml / context_memory.txt
CLI	cli.py	347	init / tick / status / recall / modify / auto-tick / load-context / append-diary

510 51034

- 1 **CLI**34734Python——CLI60-80
JSON"AI"
- 2 9534memoryPython——fileioheart.yamlcontext_memory.
txt——heart.yaml
- 3 6834311Character__init__4——34tick

34510510"——1472510
poolEE

1472510"tick"

5.3

34NumPy——"

346"——"

34

"34"34"cosine_sim3import1
38

"34EE——"——
poolEE——34"——"

5.4

34——intensity

"AI——intensityAI——
——"

[illegible]

6. ☐ ☐

[illegible]

"34" "—" "—"
"20"—
"

```
000000000000340000000000——000→00000000→pool+memories0000→tick00000000→0000—  
—0000000000000000000000000000000000000000000000000
```

34□□□□□□□□□□□□□□□□□□□□□□——□□□□□□□□□□□□□□□□□□□□□□"□□□"

7. □□

[illegible]

□ □

□ □ □ □

- 1 Bakker, A. B., & Demerouti, E. (2007). The Job Demands-Resources model: State of the
art. *Journal of Managerial Psychology*, 22(3), 309-328.
- 2 Collins, R. (2004). *Interaction Ritual Chains*. Princeton University Press.
- 3 Goffman, E. (1959). *The Presentation of Self in Everyday Life*. Anchor Books.
- 4 Hobfoll, S. E. (1989). Conservation of resources: A new attempt at conceptualizing stress.
American Psychologist, 44(3), 513-524.
- 5 Hochschild, A. R. (1983). *The Managed Heart: Commercialization of Human Feeling*.
University of California Press.
- 6 Horton, D., & Wohl, R. R. (1956). Mass communication and para-social interaction:
Observations on intimacy at a distance. *Psychiatry*, 19(3), 215-229.
- 7 Snyder, M. (1974). Self-monitoring of expressive behavior. *Journal of Personality and
Social Psychology*, 30(4), 526-537.
- 8 Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive
Science*, 12(2), 257-285.
- 9 田田. (2006). 田田田田. 田田田田.

2026 6 28 34 v7